

Q1: What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

MapReduce is slow because it stores data on the disk after every step. This increases processing time. Spark is faster because it keeps data in memory (RAM), reducing disk usage. It also supports machine learning, SQL, and real-time processing, making it a better choice for big data.

Q2: Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

Spark stores data in memory (RAM) instead of reading it from the disk every time. This makes repeated tasks like machine learning much faster because the same data can be reused without loading it again and again.

Q3: Write a code snippet to remove all duplicate rows from a DataFrame based on user_id and transaction_date.

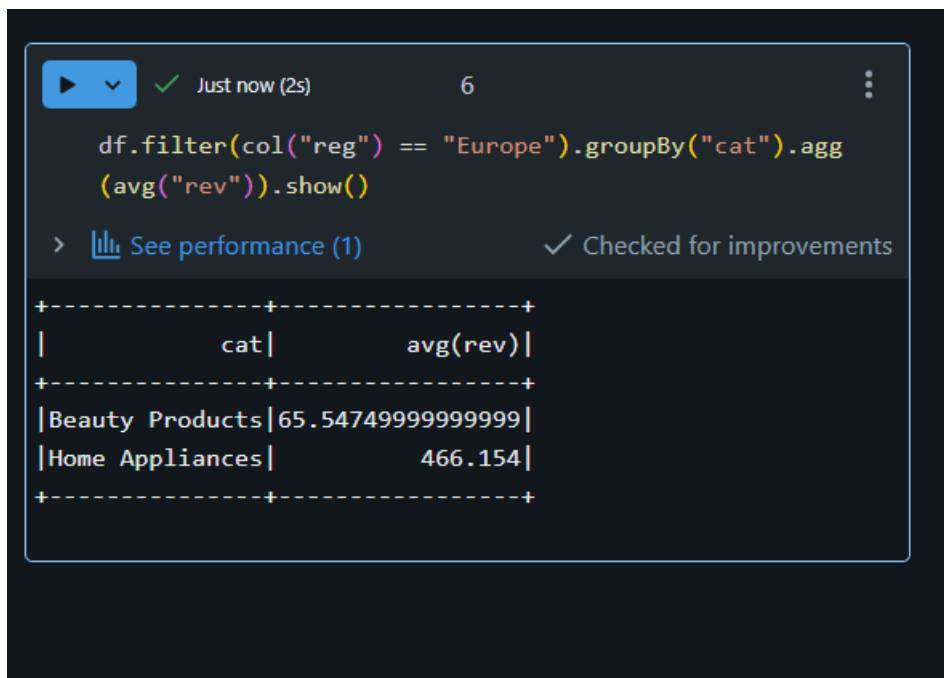
```
df1 = df.dropDuplicates(["tid", "dt"])
df1.show()
```

> [See performance \(1\)](#) [Optimize](#)

> df1: pyspark.sql.connect.dataframe.DataFrame = [tid: integer, dt: date ... 7 more fields]

tid	dt	cat	prod	qty	price	rev	reg	pay
10002	2024-01-02	Home Appliances	Dyson V11 Vacuum	1	499.99	499.99	Europe	PayPal
10011	2024-01-11	Beauty Products	Chanel No. 5 Perfume	1	129.99	129.99	Europe	PayPal
10021	2024-01-21	Clothing	Zara Summer Dress	3	59.99	179.97	Asia	Debit Card
10029	2024-01-29	Beauty Products	MAC Ruby Woo Lips...	1	29.99	29.99	Europe	PayPal
10038	2024-02-07	Home Appliances	Roomba i7+	2	799.99	1599.98	Europe	PayPal
10065	2024-03-05	Beauty Products	Lancome La Vie Es...	1	102.0	102.0	Europe	PayPal
10201	2024-07-19	Clothing	Gap 1969 Original...	3	59.99	179.97	Asia	Debit Card
10213	2024-07-31	Clothing	Uniqlo Airism Sea...	4	9.9	39.6	Asia	Debit Card
10232	2024-08-19	Books	Dune by Frank Her...	2	9.99	19.98	North America	Credit Card
10233	2024-08-20	Beauty Products	Fresh Sugar Lip T...	1	24.0	24.0	Europe	PayPal
10018	2024-01-18	Sports	Manduka PRO Yoga Mat	4	119.99	479.96	Asia	Credit Card
10019	2024-01-19	Electronics	Garmin Forerunner...	2	499.99	999.98	North America	Credit Card
10027	2024-01-27	Clothing	Adidas Ultraboost...	2	179.99	359.98	Asia	Debit Card
10035	2024-02-04	Beauty Products	L'Oreal Revitalif...	2	39.99	79.98	Europe	PayPal
10045	2024-02-14	Clothing	Patagonia Better ...	2	139.99	279.98	Asia	Debit Card
10085	2024-03-25	Electronics	Ring Video Doorbell	1	99.99	99.99	North America	Credit Card
10097	2024-04-06	Electronics	Bose SoundLink Re...	3	299.99	899.97	North America	Credit Card
10109	2024-04-18	Electronics	Google Nest Hub Max	2	229.99	459.98	North America	Credit Card

Q4: Given a DataFrame df_sales, filter rows where the region is 'West' and group by product_category to find the average sale_amount.



The screenshot shows a Jupyter Notebook cell with a Spark SQL query and its output. The query filters for the region 'Europe' and calculates the average revenue by product category. The output is a table with two columns: 'cat' and 'avg(rev)'.

```
df.filter(col("reg") == "Europe").groupBy("cat").agg(
  avg("rev")).show()
```

> [See performance \(1\)](#) ✓ Checked for improvements

cat	avg(rev)
Beauty Products	65.54749999999999
Home Appliances	466.154

Q6: Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.

No rows returned because no region has more than 100 records.

Q7: How does the immutability of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Spark DataFrames are **immutable**, which means the original DataFrame cannot be changed. When we drop or rename columns, Spark creates a **new DataFrame** instead of modifying the existing one. This helps keep the original data safe.

Q8: Write a Spark command to filter a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

```
df = df.withColumn("age", lit(25))
df = df.withColumn("subscription", lit("Premium"))
df.filter((col("age") >= 18) & (col("age") <= 30) & (col("subscription") == "Premium")).show()
```

> [See performance \(1\)](#) ✓ Checked for improvements

> df: pyspark.sql.connect.dataframe.DataFrame = [tid: integer, dt: date ... 9 more fields]

tid	dt	cat	prod	qty	price	rev	reg	pay	age	subscription
10001	2024-01-01	Electronics	iPhone 14 Pro	2	999.99	1999.98	North America	Credit Card	25	Premium
10002	2024-01-02	Home Appliances	Dyson V11 Vacuum	1	499.99	499.99	Europe	PayPal	25	Premium
10003	2024-01-03	Clothing	Levi's 501 Jeans	3	69.99	209.97	Asia	Debit Card	25	Premium
10004	2024-01-04	Books	The Da Vinci Code	4	15.99	63.96	North America	Credit Card	25	Premium
10005	2024-01-05	Beauty Products	Neutrogena Skinca...	1	89.99	89.99	Europe	PayPal	25	Premium
10006	2024-01-06	Sports	Wilson Evolution ...	5	29.99	149.95	Asia	Credit Card	25	Premium
10007	2024-01-07	Electronics	MacBook Pro 16-inch	1	2499.99	2499.99	North America	Credit Card	25	Premium

Q9: When cleaning a dataset, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?

It is better to handle null values first because they can affect the accuracy of calculations like sum() and avg(). Filling or removing null values helps produce correct and reliable results.

Q10: Write the code to revise a column named raw_timestamp by casting it to a TimestampType and renaming it to event_time.

```
dt = df.withColumn("raw_timestamp", col("raw_timestamp").cast(TimestampType()))
df = df.withColumnRenamed("raw_timestamp", "event_time")
df.select("dt", "event_time").show()
```

> [See performance \(1\)](#) ✓ Checked for improvements

> df: pyspark.sql.connect.dataframe.DataFrame = [tid: integer, dt: date ... 8 more fields]

dt	event_time
2024-01-01	2024-01-01 00:00:00
2024-01-02	2024-01-02 00:00:00
2024-01-03	2024-01-03 00:00:00
2024-01-04	2024-01-04 00:00:00
2024-01-05	2024-01-05 00:00:00
2024-01-06	2024-01-06 00:00:00
2024-01-07	2024-01-07 00:00:00
2024-01-08	2024-01-08 00:00:00
2024-01-09	2024-01-09 00:00:00
2024-01-10	2024-01-10 00:00:00
2024-01-11	2024-01-11 00:00:00
2024-01-12	2024-01-12 00:00:00
2024-01-13	2024-01-13 00:00:00
2024-01-14	2024-01-14 00:00:00
2024-01-15	2024-01-15 00:00:00
2024-01-16	2024-01-16 00:00:00
2024-01-17	2024-01-17 00:00:00
2024-01-18	2024-01-18 00:00:00

Q11: Explain the "Shuffle" process that occurs during a grouping operation. Why is it considered a wide transformation?

Shuffle is the process of moving data between different partitions while performing operations like `groupBy()`. It is called a **wide transformation** because data from multiple partitions is exchanged, which makes the operation slower than narrow transformations.

Q12: Write a code snippet that identifies and removes rows where the email column contains null values OR the username is an empty string.

```
df = df.withColumn("email", lit("user@gmail.com"))
df = df.withColumn("username", lit("user"))

df = df.filter(col("email").isNotNull() & (col("username") != ""))
df.select("email", "username").show()
```

> [See performance \(1\)](#) ✓ Checked for improvement

```
> df: pyspark.sql.connect.dataframe.DataFrame = [tid: integer, dt: date ... 10 more fields]
|user@gmail.com| user|
|user@gmail.com| user|
|user@gmail.com| user|
|user@gmail.com| user|
|user@gmail.com| user|
|user@gmail.com| user|
|user@gmail.com| user|
|user@gmail.com| user|
|user@gmail.com| user|
|user@gmail.com| user|
|user@gmail.com| user|
|user@gmail.com| user|
|user@gmail.com| user|
|user@gmail.com| user|
|user@gmail.com| user|
|user@gmail.com| user|
|user@gmail.com| user|
|user@gmail.com| user|
|user@gmail.com| user|
+-----+-----+
only showing top 20 rows
```

All rows contain valid email and username values, so no rows were removed.

Q13: How do you use the `.agg()` function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

```
from pyspark.sql.functions import *

df.agg( min("price").alias("Min Price"),max("price").alias("Max Price"), avg("price").alias("Avg Price")
).show()
```

> [See performance \(1\)](#) ✓ Checked for improvements

Min Price	Max Price	Avg Price
6.5	3899.99	236.39558333333292

Q14: In the context of cleaning a dataset, what is the risk of using inferSchema=true when your source data contains messy or inconsistent date formats?

If the date formats are inconsistent, inferSchema=true may detect the wrong data type or treat dates as strings. This can cause errors or incorrect results during data processing.

Q15: Write a final processing pipeline that:

1. Filters out duplicates.
2. Fills null prices with 0.
3. Groups by store_id to calculate total revenue.

```
df = df.withColumn("store_id", lit(1))
df = df.dropDuplicates()
df = df.na.fill(0, ["price"])
df.groupBy("store_id").sum("rev").show()
```

> [See performance \(1\)](#) ✓ Checked for improvements

> df: pyspark.sql.connect.dataframe.DataFrame = [tid: integer, dt: date ... 11 more fields]

store_id	sum(rev)
1	80567.85000000008